

LoopFuzz: Closed-Loop Runtime Admission Control for LLM-Guided Stateful Protocol Fuzzing

Ketang Chen^a, Xiaolei Ren^{a,*}

^a*Macau University of Science and Technology, Avenida Wai Long, Taipa, Macau, China*

Abstract

Stateful protocol fuzzing is difficult because a syntactically valid message can still violate the current session state. This failure mode is amplified when a large language model (LLM) can inject protocol messages into a long-running campaign. We present LoopFuzz, a closed-loop runtime admission-control design for LLM-guided stateful protocol fuzzing. LoopFuzz treats model outputs as bounded hypotheses rather than queue entries. It derives message constraints from RFC fragments, seed traces, and observed responses. It admits an intervention only when runtime evidence supports parseability, response-grounded acceptability, state reachability, and downstream gain. Failed executions become counterexamples for local refinement, and state-aware scheduling triggers model assistance only under adaptive plateau conditions. Re-audited run summaries show that LoopFuzz achieves the highest mean absolute branch coverage on eight of nine off-the-shelf targets and the highest mean IPSM edge count on all nine against AFLNet and ChatAFL. Auxiliary comparisons are mixed: NS-Fuzz leads absolute branch coverage on five of eight targets, and MQTT remains separately qualified.

Keywords: Protocol fuzzing, Large language models, Runtime admission control, Stateful testing, Counterexample-guided refinement

1. Introduction

Network protocol implementations remain difficult security targets because correctness depends on interaction history rather than isolated inputs [1–5]. An FTP server, for example, rejects a well-formed `PASS` command if no valid `USER` command has established the login state. Unlike file fuzzing, protocol fuzzing must preserve message syntax, temporal order, and request-response progress across a multi-step session [5–10].

*Corresponding author: Xiaolei Ren. Email address: xlren@must.edu.mo

Email addresses: 3250006913@student.must.edu.mo (Ketang Chen), xlren@must.edu.mo (Xiaolei Ren)

Traditional coverage-guided fuzzers work well for stateless parsers. They fit network servers poorly when later behavior depends on earlier valid exchanges [11–15]. Stateful greybox fuzzers such as AFLNet and StateAFL partially close this gap. They infer progress from responses and replay valid traces [1–4, 7, 16, 17]. Even so, byte-level mutation still struggles to synthesize the context-sensitive messages needed to cross deep protocol boundaries.

Large language models offer a plausible source of such messages. ChatAFL and later LLM-assisted fuzzers show that a model can extract structure from specifications, logs, or documentation and produce fluent candidates [18–25]. But fluency is not enough. A request can be grammatically plausible and still be invalid for the live session state. Such executions spend budget without improving queue quality.

LoopFuzz places a runtime admission controller between LLM generation and queue admission. It treats the LLM as a hypothesis generator rather than an oracle. It derives message hypotheses from available RFC fragments, seed traces, and observed responses. It stores failed executions as counterexamples and uses them for counterexample-guided refinement (CEGR) rather than unconstrained regeneration. It also uses inferred protocol state machine (IPSM) feedback to prioritize frontier states and rare transitions, and to trigger assistance only when ordinary mutation stalls. The name *LoopFuzz* emphasizes closed-loop admission and refinement, avoiding any suggestion of formal verification of protocol semantics, model behavior, or server correctness.

The contributions of this paper are as follows:

- **Hypothesis-Driven Message Modeling.** LoopFuzz derives message grammars and field constraints from RFC fragments, seed traces, and observed responses, and maintains them as explicit runtime hypotheses.
- **A Closed-Loop Runtime Admission Boundary.** LoopFuzz inserts a runtime admission controller between the LLM and the queue. The controller checks parseability, response-grounded acceptability, state reachability, and downstream gain before admission.
- **Counterexample-Guided Local Refinement.** LoopFuzz preserves failed request-response traces as counterexamples and uses them to patch one field constraint or production at a time, which constrains model freedom and stabilizes refinement.
- **State-Aware Scheduling.** LoopFuzz leverages IPSM topology, state productivity, and adaptive plateau detection to prioritize underexplored frontier states and to request model assistance only when mutation alone stalls.
- **An Empirical Study of Runtime-Validated Assistance.** We evaluate LoopFuzz on a repeated-run benchmark archive. The study includes nine off-the-shelf primary comparisons against AFLNet and ChatAFL using final `b_abs` and `edges`, auxiliary four-way NSFuzz comparisons on

eight targets, a separately qualified MBFuzzer/LoopFuzz Mosquitto comparison, and policy ablations of the evaluated controller.

Accordingly, the paper evaluates LoopFuzz as a campaign-control policy, not as a standalone message generator. In stateful protocol fuzzing, generation matters only when it survives runtime evidence and improves downstream queue quality.

2. Background and Motivation

Table 1: Among the listed systems, LoopFuzz combines LLM hypotheses with runtime admission and online refinement; representative prior fuzzers differ in input prior, state signal, and admission boundary.

System	Input prior	State signal	Admission rule	Online refinement
Coverage-guided mutation fuzzing [11, 12]	Byte mutation	None	None	No
AFLNet [1]	PCAP mutation	Response	Novelty	No
StateAFL [2]	PCAP mutation	Memory	Novelty	No
ChatAFL [18]	LLM requests	History text	Direct admission	No
LoopFuzz (ours)	LLM hypotheses	IPSM frontier + validation	Runtime admission	Yes

Table 1 contrasts LoopFuzz with representative fuzzers. Prior LLM-assisted protocol fuzzers improve input fluency, but they leave admission largely open-loop. We study a narrower question: does closed-loop runtime admission control make LLM proposals more useful in a long-running stateful campaign?

2.1. Coverage-Guided Fuzzing vs. Network Targets

A coverage-guided fuzzer operates through a repeated feedback cycle of seed selection, mutation, execution, coverage measurement, and novelty-based queue update [12–15]. This loop is effective for file parsers, but network targets impose three constraints that matter here:

1. **Temporal state dependence:** server behavior depends on the order and semantics of earlier messages, not only on the current payload [1–4].
2. **Strict message format:** protocols require delimiters, field layouts, and often checksum or length consistency, so many mutated messages die before they reach deeper logic [4, 26, 27].
3. **Execution instability:** sockets, forking daemons, and asynchronous response handling complicate both coverage collection and campaign throughput [17].

2.2. Stateful Greybox Fuzzing (SGF)

SGF addresses this gap by treating the target as an *inferred protocol state machine* (IPSM). Formally, the IPSM can be viewed as a Mealy machine $M = (Q, I, O, \delta, \lambda, q_0)$, where:

- Q is the set of inferred protocol states;

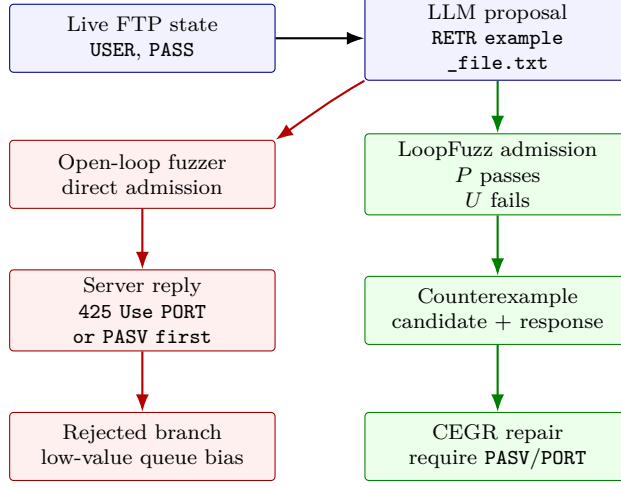


Figure 1: Semantic drift in an FTP session. The model proposes `RETR` before the session opens a data channel. Open-loop fuzzing admits the rejected branch. LoopFuzz rejects the candidate, records the 425 counterexample, and repairs the missing `PASV/PORT` precondition.

- I is the set of client messages;
- O is the set of observed server responses;
- δ maps a state and input to a successor state;
- λ maps a state and input to an observed response; and
- q_0 is the initial connection state.

AFLNet-style fuzzers infer Q from observed responses and use that structure to bias later exploration toward rare or promising parts of the protocol. Here, the IPSM matters for one reason. It gives LoopFuzz a notion of progress that is richer than raw edge coverage alone.

2.3. LLM Integration into Protocol Fuzzing

Large language models make it easier to propose structure-aware protocol messages from RFC text, interaction history, or observed responses. That helps with syntax and message diversity, but it does not solve the harder control problem: a fluent request can still be wrong for the live session state.

2.4. A Motivating Example: Semantic Drift

Consider an FTP fuzzing campaign against ProFTPD, as illustrated in Fig. 1. The fuzzer executes `USER` and `PASS`. It then queries the LLM for the next request. The model returns `RETR example_file.txt`. The request is syntactically fluent but semantically premature. The client lacks an active data channel via `PASV` or `PORT`. The server rejects it with `425 Use PORT or PASV first`.

We call this failure *semantic drift*. The request is locally plausible, but it violates the precondition of the current session state. An open-loop LLM fuzzer admits the request because it looks fluent. It then mutates a rejected branch instead of crossing the missing state transition. LoopFuzz blocks this drift at the admission boundary. The controller treats the 425 reply as negative evidence. It rejects the candidate and records the failed trace as a counterexample. CEGR then repairs the missing PASV/PORT precondition instead of regenerating the full continuation. This example captures the paper’s central claim: the value of LLM assistance depends on runtime admission, not on surface fluency. We use semantic drift as an operational motivation for rejection and CEGR repair, not as a separately estimated global rate.

3. Related Work

We position LoopFuzz against the prior lines that are closest in mechanism and evaluation.

Earlier structure-aware fuzzing used explicit message structure, inferred snippets, token constraints, or grammar-level guidance to preserve validity [26–32]. These systems improve input quality. They do not address the control problem that arises when an external model proposes state-dependent protocol messages.

Stateful greybox and state-aware network-service fuzzing address protocol interaction more directly. AFLNet, StateAFL, NSFuzz, and the broader SGF line infer or expose state structure from server responses, process memory, or program variables [1–3, 16, 33]. They then use that structure to prioritize rare or productive traces.

Later systems extend this idea to packet-sequence exploration, format-sensitive protocol fuzzing, and communication-heavy environments [4–8, 34–38]. LoopFuzz builds on this line, but adds a runtime-validated admission boundary for LLM interventions. SGF, NSFuzz, Bleem, Logos, and MBFuzzer are therefore important external comparison points beyond AFLNet and ChatAFL. The present evaluation archive does not include direct head-to-head reproduction against that broader set, so we treat that omission as a limitation rather than as evidence that those systems are weaker.

Adjacent work studies state-aware or interaction-heavy fuzzing in robotic policies, firmware, Linux drivers, data-distribution middleware, hardware, and embedded systems [39–45]. These systems are not protocol fuzzers in the narrow sense. They still support the broader point that state feedback and interaction context can dominate raw mutation quality.

Target-aware scheduling and semantic validation form another relevant line. Directed greybox fuzzing studies how to steer search toward high-value targets [11, 46, 47]. Work on seed selection and strategy adaptation studies how to spend budget more effectively [12–15]. Semantic validation and fuzz-driver generation show that stronger inputs often require an explicit acceptance interface [48, 49]. LoopFuzz inherits that lesson and applies it to LLM proposals.

Related work also studies how to infer dependencies or exploit guidance from data-flow and learned signals outside protocol fuzzing proper [50–52]. This line

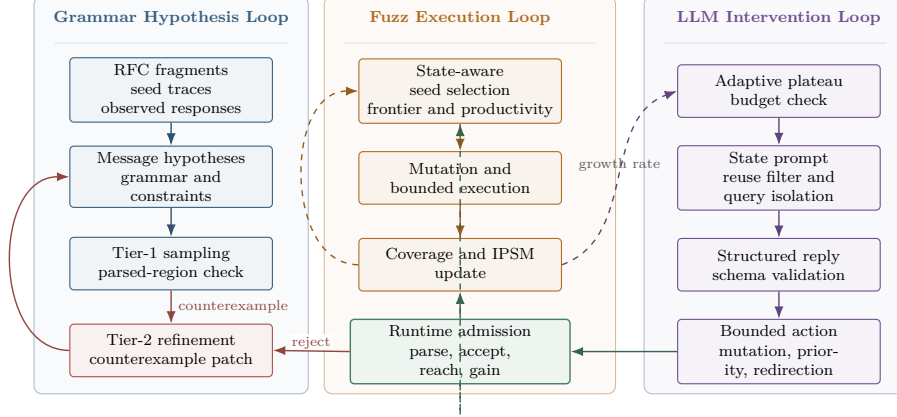


Figure 2: LoopFuzz changes the LLM from a queue producer into a bounded policy proposer. The Grammar Hypothesis Loop maintains message hypotheses, the Fuzz Execution Loop updates coverage and IPSM feedback, and the LLM Intervention Loop proposes bounded interventions; runtime admission checks mediate each proposal before it can affect the queue.

reinforces our decision to treat the LLM as a bounded proposal mechanism rather than as an unconstrained generator.

LLM guidance addresses the synthesis bottleneck more directly. ChatAFL uses RFCs and interaction histories to generate protocol messages [18]. Adjacent systems use LLMs for general fuzz input generation and documentation-guided constraint extraction [19, 20, 22, 53]. Other systems use LLMs for specification-guided device fuzzing or fuzz-driver generation [21, 23]. Our work is closest to ChatAFL. The difference is the control boundary. LoopFuzz asks whether a proposal should be admitted, whether its hypothesis survives runtime evidence, and how the IPSM should guide the next query.

Benchmarking practice is also part of the context. ProFuzzBench established a reusable baseline for repeated protocol-fuzzing experiments [16]. Recent methodology work shows how easily comparisons become unstable without fixed budgets, repeated trials, and careful interpretation of coverage signals [54–56]. We follow that guidance because runtime-validated assistance is especially sensitive to campaign history.

4. LoopFuzz Design

Fig. 2 summarizes the three coupled loops in LoopFuzz: the Grammar Hypothesis Loop, the Fuzz Execution Loop, and the LLM Intervention Loop.

LoopFuzz preserves the stateful greybox fuzzing execution path. It sets a strict control boundary between LLM generation and queue admission. A model output changes the campaign only after runtime evidence supports admission. The admission rule checks message structure, parseability, response behavior, inferred state reachability, and downstream campaign value. The method has

four coupled components: grammar hypothesis generation, runtime admission control, counterexample-guided refinement, and state-aware scheduling.

This control boundary is the core idea of the paper. ChatAFL is open-loop: it uses a fixed trigger, a plain-text prompt, and direct admission of the extracted request. LoopFuzz is closed-loop: IPSM growth, state productivity, and hypothesis fitness determine when the LLM is queried, what context it receives, and whether its output is admitted.

The three loops in Fig. 2 define the analytical roles of the method. The Grammar Hypothesis Loop constructs and repairs message hypotheses from RFC knowledge, seed traces, and observed responses. The Fuzz Execution Loop performs state selection, mutation, bounded execution, IPSM update, and novelty checking. The LLM Intervention Loop is activated only under plateau conditions and returns either a validated continuation or a bounded control action.

The remaining labels in Fig. 2 denote control interfaces rather than independent claims. *Tier-1 Sampling* is the cheap online check over parsed execution regions. *Tier-2 Refinement* is the counterexample-triggered repair path. *Schema validation* checks the structure of a model reply before bounded action dispatch. *Bounded action dispatch* maps an admitted action to one of three campaign controls: mutation proposal, seed prioritization, or target-state redirection. The *prompt reuse filter* prevents repeated prompts over the same state summary. Query isolation bounds failures in the model-query path.

4.1. Grammar Hypothesis Loop

Let $\mathcal{L}(c, s, \Sigma)$ denote the LLM generation function under prompt context c , state summary s , and protocol knowledge Σ . LoopFuzz uses $\mathcal{L}$ in two stages. During initialization, it preserves ChatAFL’s template extraction and seed enrichment path. Initial enrichment is scheduled as a bounded phase so that corpus diversification does not dominate early campaign time. During fuzzing, LoopFuzz adds a lazily initialized hypothesis layer. This layer is activated at the first plateau event and updated through sampled validation and refinement. The separation is important. The initialization phase broadens the initial seed set. Runtime hypotheses support runtime-validated control during long campaigns.

The Grammar Hypothesis Loop draws on three evidence sources when they are available: RFC fragments, queued seed traces or packet examples, and observed server responses, including response codes and error texts. For each message type m , the loop maintains a hypothesis $h_m = (G_m, \Phi_m)$. Here, G_m is an ABNF-style or template-style production, and Φ_m is a set of field constraints. In LoopFuzz, Φ_m captures length bounds, enumerated values, token dependencies, and simple cross-field relations. This representation is more constrained than free-form text and lighter than a full formal protocol specification. We choose it because sampled online validation must remain cheap.

The evidence sources are not assumed to be complete. RFC extraction supplies useful protocol structure when a relevant fragment is available. Seed traces provide executable examples of message order and framing. Observed

responses provide target-specific evidence about rejection, progress, and state change. This division is important for the claim. LoopFuzz does not require a full protocol specification. It requires enough evidence to form testable hypotheses and then lets runtime admission decide whether those hypotheses should affect the campaign.

4.2. Runtime Admission Controller

The runtime admission controller is the central mechanism in our design. It does not formally prove correctness. Instead, it enforces four runtime checks before an LLM contribution can affect the campaign: parseability, acceptability, state reachability, and coverage gain. A candidate intervention x is admitted only if

$$A(x) = P(x) \wedge U(x) \wedge R(x) \wedge G(x),$$

where P denotes local parseability, U denotes response-grounded acceptability, R denotes evidence of new or redirected state reachability, and G denotes downstream gain in IPSM structure or coverage.

The FTP example in Fig. 1 illustrates the same checks. The candidate `RETR example_file.txt` can satisfy local syntax under P , but the live 425 response makes U fail because no data channel has been negotiated. The failed execution also provides no useful new transition or coverage signal, so R and G do not support admission. LoopFuzz therefore rejects the candidate and records a counterexample that constrains CEGR to repair the missing PASV/PORT precondition.

4.2.1. Structured Output and Parseability

The plateau stage accepts either a concrete next-request candidate or a bounded structured-action object. Runtime validation first checks structural well-formedness, field types, length bounds, and action-schema consistency. LoopFuzz then tests local parseability against the current message hypotheses. It reuses parsed message regions already available during ordinary execution. This step rejects structurally invalid interventions before they perturb campaign control.

4.2.2. Response-Grounded Acceptability

Parseability alone is insufficient. A parsed request can still be semantically premature. LoopFuzz therefore classifies the resulting response and treats a non-rejection response as positive evidence. Rejected or persistently unproductive responses reduce state productivity and hypothesis fitness. This gives an execution-grounded notion of acceptability rather than a semantic proof. The controller asks a narrow question: does live target behavior support admission?

4.2.3. State Reachability and Coverage Gain

The final admission question is whether a candidate helps the campaign. LoopFuzz measures this with the standard AFLNet-style novelty test on new

IPSM states, new IPSM transitions, or new edge coverage. The controller therefore uses an IPSM-derived reachability signal, not text quality alone. A hypothesis fragment remains in active use only when execution provides positive evidence in state reachability or coverage. Fluent but repeatedly unproductive candidates are filtered out at the control boundary instead of accumulating as low-value queue bias.

4.3. Counterexample-Guided Refinement

When sampled validation fails repeatedly, LoopFuzz records a concrete counterexample. The record contains the failure-triggering request fragment, the observed response, and the mismatch against the current hypothesis. Refinement is deliberately local. Instead of rewriting the full protocol description, the refinement interface constrains each repair to one field constraint or one production fragment. This single-patch discipline narrows the model’s search space, limits unconstrained edits, and makes the repair path auditable. The resulting update is reintegrated into protocol matching and re-evaluated by later sampled validation.

This repair boundary is intentionally weaker than formal grammar repair. The one-field or one-production constraint is enforced through the refinement prompt and audit log, not through hard AST differencing. The current implementation also stores concrete failed traces without counterexample minimization. We therefore claim bounded local refinement, not complete protocol inference.

4.4. State-Aware Scheduling and Plateau-Triggered Assistance

Reducing malformed generations solves only half of the problem. Stateful fuzzers also stagnate when the campaign keeps revisiting locally dense IPSM regions while failing to cross narrow protocol bridges. LoopFuzz therefore treats IPSM feedback as a first-class scheduling signal.

To address this, LoopFuzz augments state selection with a frontier bonus β derived from the out-degree of the IPSM:

$$\beta = \begin{cases} 4.0 & \text{if } \deg^+(s) \leq 1 \\ 2.0 & \text{if } \deg^+(s) \leq 3 \\ 1.0 & \text{otherwise} \end{cases}$$

States absent from the IPSM receive the maximum bonus $\beta=4$. LoopFuzz then modulates this bonus with a three-level error-productivity hint: unknown, likely error, or confirmed productive. The reported implementation encodes these levels as 0, 1, and 2. This hint downweights frontier states that repeatedly behave like rejection sinks, but it does not discard them outright.

In the reported policy, hint level 1 applies a $\beta \times 0.5$ penalty. Hint level 0 is reduced to $\beta \times 0.7$ only after repeated low-productivity selections (*selected_times* ≥ 30 and *productivity* < 0.005). Hint level 2 receives no penalty. The hint is initialized from response-code classification and updated online. As a result, states can be

promoted or demoted as evidence accumulates. They are not fixed by an early response pattern. This policy approximates a rare-transition preference without requiring an exact model of the full protocol state space.

These constants should be read as the reported control policy, not as a claim of global optimality. The three frontier-bonus levels $\{4.0, 2.0, 1.0\}$ separate unseen or low-out-degree frontier states from denser regions by coarse preference rather than by fine-grained tuning. The error-productivity penalties and the $(30, 0.005)$ gate reduce repeated investment in response-derived rejection sinks. They still allow those states to recover if later executions become productive. Likewise, the $(80, 30\%)$ node-stagnation safeguard and the 64-call intervention budget are bounded control parameters. They prevent persistent hot-spot lock-in and unbounded LLM overhead. We report them to make the policy auditable. This study does not claim a full sensitivity analysis over the entire hyperparameter surface.

A complementary graph stagnation check tracks whether IPSM growth has collapsed at the graph level rather than only at the local state level. When no new nodes appear for an extended interval, the scheduler intermittently injects random selection to avoid persistent hot-spot lock-in. In the reported configuration, that safeguard activates after 80 consecutive rounds without node growth and uses a 30% random-selection probability.

Plateau detection uses the IPSM edge growth rate $\dot{E}$, measured as edges gained per minute. LoopFuzz recalculates this rate every 60 seconds of wall-clock time and uses it to set an adaptive threshold:

$$\tau(\dot{E}) = \begin{cases} 300 & \text{if } \dot{E} > 5 \\ 200 & \text{if } 1 < \dot{E} \leq 5 \\ 150 & \text{if } \dot{E} \leq 1 \end{cases}$$

The initial value is $\tau=200$ for textual protocols. MQTT uses the higher thresholds $\{500, 400, 300\}$ under the same edge-growth bins because binary framing and broker-mediated behavior expose state progress more sparsely. More generally, LoopFuzz treats plateau triggering as a rate-sensitive control decision, not as a fixed-frequency query rule.

Assistance is invoked only when edge growth remains weak and the model-intervention budget is not exhausted. In the reported configuration, the plateau path opens when the unproductive streak reaches τ and prior model interventions remain below 64. Once inside that path, hypotheses are initialized lazily on first use. Refinement remains gated by validation count and accumulated counterexamples. The prompt reuse filter covers the most recent 64 LLM invocations. This prevents assistance from reacting to the same state summary again and again.

The plateau prompt exposes a compact state summary rather than a raw execution log. It includes IPSM node and edge counts, the current edge-growth rate, pending favored seeds, queued paths, the current target state, and the least productive states. The admissible action space is also bounded. A model can redirect the target state, reprioritize a small set of seeds, or propose bounded

Algorithm 1: Runtime-Validated Plateau Recovery in LoopFuzz

Input: Target T , LLM interface $\mathcal{L}$, corpus C , edge-growth signal $\dot{E}$
Output: Coverage bitmap $\mathcal{B}_{global}$ and IPSM $\mathcal{M}$

```
1 Initialize  $\mathcal{B}_{global}$  and  $\mathcal{M}$ 
2 while the campaign budget remains do
3    $s \leftarrow \text{SELECTSEED}(C, \mathcal{M})$ 
4    $(trace, Q) \leftarrow \text{MUTATEEXECUTE}(T, s)$ 
5   Update  $\mathcal{B}_{global}$ ,  $\mathcal{M}$ , and frontier scores from  $(trace, Q)$ 
6   if sampled validation is due then
7     Validate active hypotheses on parsed regions of  $Q$ 
8   end
9   if the adaptive plateau gate opens then
10    Initialize hypotheses on first use, if needed
11    if counterevidence opens the refinement gate then
12      Patch the lowest-fitness hypothesis locally
13    end
14     $c \leftarrow \text{BUILDPROMPT}(\mathcal{M}, C, Q, \dot{E})$ 
15    if  $c$  is not in the recent prompt cache then
16       $r \leftarrow \mathcal{L}(c)$  under process isolation
17      Admit  $r$  only if parseability, acceptability, reachability, and gain all hold
18      Apply the admitted bounded action to  $C$ 
19    end
20  end
21 end
```

mutations over an existing seed. These actions are not admitted because they are fluent. They must still pass the same runtime admission boundary. This design keeps the LLM in the role of a state-directed policy proposer.

MQTT-specific scheduling is treated as a protocol adaptor, not as the general mechanism of LoopFuzz. For MQTT campaigns, packet-type selection uses a state-indexed action policy, and mutation-family selection uses a bandit-style controller. These signals help expose broker forwarding behavior to the IPSM, but the paper reports Mosquitto separately because this adaptor is protocol-specific and because MBFuzzer is a specialized MQTT baseline. In short, assistance is conditional, budgeted, state-directed, and scoped by protocol evidence.

LoopFuzz also supports protocol-adaptor signals that are not used as headline evidence in the primary table. For MQTT, the adaptor translates between binary packets and textual hypotheses, drives multi-party broker interactions, and records differential behavior across broker-facing roles. A lightweight protocol oracle can log semantic violations such as authentication bypass, state-order violations, information leaks, traversal patterns, injection patterns, or denial-of-service indicators. These signals are useful for auditing and for later bug triage. They are not treated as confirmed vulnerabilities in this paper because crash deduplication, root-cause analysis, exploitability review, and cross-target oracle calibration are not normalized across the archive.

Algorithm 1 specifies the plateau path and the hypothesis-refinement gate. It formalizes how state-aware triggering, runtime admission checks, and CEGR interact within one campaign loop.

This integration also imposes a bounded operating model. The initial enrichment phase is separated from long-running admission control so that seed completion does not dominate early wall-clock time. Hypothesis construction is deferred until the first plateau event. This avoids paying the full refinement cost before the campaign exposes evidence of stagnation. Model queries use process isolation, and network cleanup is bounded to limit persistent stalls on forking daemons. These mechanisms are not separate novelty claims. They define the operating conditions under which runtime admission remains compatible with long-running stateful fuzzing.

5. Experimental Evaluation Design

We organize the evaluation around three research questions that match the paper’s main claims. We ask whether closed-loop admission control changes whole-campaign behavior, whether the evaluated state-aware control policy helps campaigns escape IPSM plateaus, and whether those effects persist in final 24-hour outcomes.

- **RQ1 (Whole-Campaign Effect):** Does runtime-validated LLM assistance prevent the coverage regressions that can arise from direct open-loop generation?
- **RQ2 (IPSM Escape Dynamics):** Does the evaluated state-aware policy, with frontier scheduling and plateau-triggered intervention, sustain IPSM growth after the baselines stall?
- **RQ3 (End-of-Campaign Outcome):** Under equal 24-hour budgets, how does LoopFuzz affect final `b_abs` and final `edges` relative to prior stateful greybox baselines?

5.1. Experimental Setup

We ran all experiments on a dedicated Ubuntu 20.04 server with 32 CPU cores and 64GB RAM. The benchmark suite spanned 10 implementations across seven protocol families:

1. **FTP:** LightFTP, bftpd, ProFTPD, and Pure-FTPd.
2. **SMTP:** Exim.
3. **RTSP:** Live555.
4. **SIP:** Kamailio.
5. **DAAP/DACP:** Forked-daapd.
6. **HTTP:** Lighttpd1.
7. **MQTT:** Mosquitto.

The benchmark suite contains 10 targets, but the provided run-summary archive supports different comparisons on different subsets. The primary comparison uses AFLNet, ChatAFL, and LoopFuzz on the nine off-the-shelf non-MQTT targets: LightFTP, bftpd, ProFTPD, Pure-FTPd, Exim, Live555, Kamailio, Forked-daapd, and Lighttpd1. We retain only completed runs with at

least 1400 archived minutes. If a fuzzer-target pair contains more than 10 valid runs, we keep the lowest 10 run IDs. This rule leaves six eligible LightFTP ChatAFL runs and trims extra LoopFuzz rows for targets with more than 10 valid runs. The audit is based on the updated primary, ablation, NSFuzz, and MBFuzzer result archives.

Mosquitto needs separate treatment. In the provided archive, the Mosquitto rows do not form the same 24-hour three-way comparison as the nine primary targets. AFLNet has 10 completed 24-hour rows. The available LoopFuzz Mosquitto rows report **b_abs** and **edges**, but their runtimes range from 31 to 444 minutes, with a mean of 137.4 minutes. MBFuzzer provides a Mosquitto summary under its own **branch_covered** counter rather than the AFLNet-lineage run-summary format. We therefore report Mosquitto as a separately qualified MQTT auxiliary comparison rather than pooling it with the primary 24-hour table.

The equal-budget primary subjects are textual request/response protocols. Mosquitto is treated separately through an MQTT-specific adaptor. We restrict our claims to this regime. The method has not been validated on opaque binary or encrypted protocols such as DNS, TLS, SSH, DTLS, or DICOM.

Both ChatAFL and LoopFuzz used the same pinned **GPT-4o-mini** endpoint. Prompts remained protocol-agnostic beyond protocol name, observed state, and runtime summary. We did not hand-specialize prompt text per target. The primary table uses approximately 24-hour archived repeated runs under the deterministic selection rule above, with no per-system cherry-picking of directories or best runs. MQTT is excluded from that equal-budget primary table because the available rows do not provide a comparable three-way 24-hour archive.

5.2. Baselines, Metrics, and Statistical Protocol

We used two baseline layers. The *primary* baselines were AFLNet as the stateful greybox baseline and ChatAFL as the direct LLM-assisted baseline. The *auxiliary* reproduced baselines were NSFuzz [33] on eight non-HTTP/non-MQTT targets and MBFuzzer [6] on Mosquitto. For NSFuzz, we compare AFLNet, ChatAFL, LoopFuzz, and NSFuzz side by side using the archived **b_abs** and **edges** fields. For MBFuzzer, we compare only a coverage-like endpoint because MBFuzzer’s summary does not expose the AFLNet-lineage **b_abs** or **edges** fields.

The main dependent variables are final **b_abs** and final **edges**. The **b_abs** field is the absolute branch-coverage count in the run-summary archive. The **edges** field is the archived IPSM edge count and is used as a state-exploration diagnostic. For NSFuzz, **edges** is an artifact-level field. We do not treat it as semantically identical to the response-derived IPSM transition abstraction maintained by the primary AFLNet-lineage fuzzers.

For each primary target and metric, we summarized selected terminal values with mean and sample SD. We then applied a two-sided exact Mann-Whitney U test to the per-run terminal values. The tables report terminal aggregates only, not significance at every timestamp. We also preserved crash artifacts, but this

study does not claim novel vulnerabilities because crash triage and root-cause validation are not normalized across the suite.

5.3. Policy-Ablation Design

The ablation design follows the switches present in the provided ablation archive. The archived matrix compares the evaluated full controller against four policy perturbations: disabling CEGR, disabling frontier scheduling, disabling hypothesis validation, and disabling adaptive triggering. This organization reflects the actual archive rather than an idealized orthogonal factorization. We apply the same deterministic run-selection rule as the primary comparison: keep completed runs with at least 1400 minutes, and if more than 10 valid runs remain, keep the lowest 10 run IDs.

5.4. Trace Accounting and Cost Reporting

Because LoopFuzz is a closed-loop LLM-in-the-loop system, coverage alone is not enough for accounting. We therefore logged each plateau event with its prompt, raw model reply, and token use. We also logged each refinement event with the affected hypothesis and its counterexamples. The ProFTPD semantic-drift example in Fig. 1 is a concrete instance of this evidence pattern: candidate request, target response, admission decision, and refinement target. These logs are sufficient to recover target-level admission statistics such as parseability pass rate, response-grounded acceptance rate, and CEGR repair outcomes. Those summaries are not normalized across the full archive. We therefore report them as audit artifacts and as a limitation rather than as aggregate evidence. As a result, semantic drift remains a motivating failure mode and an auditable trace category here, not a quantified cross-target headline metric.

The four data directories audited here do not contain a normalized cross-target LLM-cost summary. We therefore do not make a cost-efficiency claim in this re-audited experimental section.

6. Results

Table 2 summarizes the terminal aggregates used for the primary comparison.

6.1. RQ1: Whole-Campaign Effect

LoopFuzz is evaluated here as an integrated control policy, not as an isolated rejection heuristic. Table 2 shows that direct open-loop assistance is not uniformly robust under the selected archived runs. ChatAFL ties AFLNet on LightFTP, finishes below AFLNet on Exim in **b_abs**, is close to AFLNet on Forked-daapd, and improves several other targets. Under these conditions, improved generation alone does not consistently translate into stronger end-of-campaign coverage.

Across the nine primary targets, LoopFuzz reaches the highest mean final **b_abs** on eight targets: LightFTP, bftpd, ProFTPD, Pure-FTPd, Exim,

Table 2: Primary end-of-campaign results for **b_abs** and **edges** (mean $\pm$ sample SD over selected completed runs). Bold marks the highest mean among AFLNet, ChatAFL, and LoopFuzz. * marks LoopFuzz **b_abs** wins significant versus both primary baselines under a two-sided exact Mann-Whitney U test at $p < 0.05$; $\dagger$ marks LoopFuzz **edges** wins significant versus both primary baselines. All rows use 10 runs per fuzzer except LightFTP ChatAFL, where the updated archive contains six eligible completed runs.

Proto.	Target	AFLNet	Final b_abs ChatAFL	LoopFuzz	AFLNet	Final edges ChatAFL	LoopFuzz
FTP	LightFTP	71.0 $\pm$ 0.0	71.0 $\pm$ 0.0	71.6$\pm$1.6	182.8 $\pm$ 10.2	200.0 $\pm$ 6.2	213.9$\pm$9.4$\dagger$
FTP	bftpd	488.6 $\pm$ 13.9	492.3 $\pm$ 15.1	501.5$\pm$7.4	166.9 $\pm$ 9.4	205.2 $\pm$ 9.7	221.7$\pm$16.9$\dagger$
FTP	ProFTPD	4919.9 $\pm$ 97.8	5317.3 $\pm$ 101.7	5358.8$\pm$136.1	180.5 $\pm$ 23.1	250.5 $\pm$ 9.1	271.7$\pm$18.8$\dagger$
FTP	Pure-FTPD	1054.3 $\pm$ 76.4	1179.2 $\pm$ 31.0	1242.6$\pm$30.6*	213.9 $\pm$ 11.9	266.2 $\pm$ 14.3	291.0$\pm$23.6$\dagger$
SMTP	Exim	3794.5 $\pm$ 42.4	3746.2 $\pm$ 47.7	3908.5$\pm$43.2*	88.3 $\pm$ 5.3	116.4 $\pm$ 10.2	128.5$\pm$7.8$\dagger$
RTSP	Live555	2894.1 $\pm$ 23.3	2911.1 $\pm$ 10.9	3088.7$\pm$22.4*	103.9 $\pm$ 10.1	151.2 $\pm$ 3.9	158.7$\pm$8.5$\dagger$
SIP	Kamailio	10244.0 $\pm$ 118.1	10314.7 $\pm$ 98.4	10402.7$\pm$141.3	101.1 $\pm$ 7.7	115.7 $\pm$ 13.9	127.6$\pm$9.6
DAAP/DACP	Forked-daapd	2326.3$\pm$77.3	2315.3 $\pm$ 50.1	2171.9 $\pm$ 226.0	21.1 $\pm$ 1.3	20.5 $\pm$ 2.1	22.1$\pm$1.5
HTTP	Lighttpd1	1930.4 $\pm$ 18.7	1987.5 $\pm$ 38.3	2156.5$\pm$111.8*	14.9 $\pm$ 3.8	29.7 $\pm$ 4.7	32.5$\pm$10.8

Table 3: Auxiliary four-way NSFuzz comparison using **b_abs** and **edges**. Bold marks the highest mean within each row and metric. NSFuzz **edges** is reported as an archived artifact field, not as a guaranteed semantically identical IPSM abstraction. All rows use the selected-run rule; LightFTP ChatAFL has six eligible runs, and the other cells have 10 selected runs.

Target	AFLNet	ChatAFL	Final b_abs LoopFuzz	NSFuzz	AFLNet	ChatAFL	Final edges LoopFuzz	NSFuzz
LightFTP	71.0 $\pm$ 0.0	71.0 $\pm$ 0.0	71.6 $\pm$ 1.6	414.3$\pm$9.8	182.8 $\pm$ 10.2	200.0 $\pm$ 6.2	213.9$\pm$9.4	11.0 $\pm$ 0.0
bftpd	488.6 $\pm$ 13.9	492.3 $\pm$ 15.1	501.5$\pm$7.4	477.6 $\pm$ 3.9	166.9 $\pm$ 9.4	205.2 $\pm$ 9.7	221.7$\pm$16.9	48.7 $\pm$ 4.8
ProFTPD	4919.9 $\pm$ 97.8	5317.3 $\pm$ 101.7	5358.8$\pm$136.1	5335.0 $\pm$ 86.9	180.5 $\pm$ 23.1	250.5 $\pm$ 9.1	271.7$\pm$18.8	186.0 $\pm$ 11.5
Pure-FTPD	1054.3 $\pm$ 76.4	1179.2 $\pm$ 31.0	1242.6 $\pm$ 30.6	1332.2$\pm$8.8	213.9 $\pm$ 11.9	266.2 $\pm$ 14.3	291.0$\pm$23.6	17.6 $\pm$ 0.5
Exim	3794.5 $\pm$ 42.4	3746.2 $\pm$ 47.7	3908.5 $\pm$ 43.2	4191.5$\pm$62.0	88.3 $\pm$ 5.3	116.4 $\pm$ 10.2	128.5$\pm$7.8	13.9 $\pm$ 1.2
Live555	2894.1 $\pm$ 23.3	2911.1 $\pm$ 10.9	3088.7$\pm$22.4	3021.5 $\pm$ 42.1	103.9 $\pm$ 10.1	151.2 $\pm$ 3.9	158.7$\pm$8.5	49.8 $\pm$ 2.0
Kamailio	10244.0 $\pm$ 118.1	10314.7 $\pm$ 98.4	10402.7 $\pm$ 141.3	10635.3$\pm$123.8	101.1 $\pm$ 7.7	115.7 $\pm$ 13.9	127.6$\pm$9.6	14.3 $\pm$ 0.7
Forked-daapd	2326.3 $\pm$ 77.3	2315.3 $\pm$ 50.1	2171.9 $\pm$ 226.0	2722.8$\pm$44.6	21.1 $\pm$ 1.3	20.5 $\pm$ 2.1	22.1 $\pm$ 1.5	27.0$\pm$6.1

Table 4: MQTT auxiliary comparison. AFLNet and LoopFuzz are taken from the provided Mosquitto run summaries; MBFuzzer is taken from its Mosquitto summary. This is not a same-budget or same-metric comparison: LoopFuzz runs are much shorter, and MBFuzzer reports **branch_covered** under its own artifact format rather than **b_abs** and **edges**.

System	Runs	Runtime (min)	Branch endpoint	Edge endpoint
AFLNet	10	1479.0 $\pm$ 0.0	b_abs : 1814.3 $\pm$ 147.3	edges : 36.0 $\pm$ 4.5
LoopFuzz	10	137.4 $\pm$ 134.7	b_abs : 2518.5 $\pm$ 269.9	edges : 107.7 $\pm$ 18.1
MBFuzzer	10	1480.0 $\pm$ 0.0	branch_covered : 2827.0 $\pm$ 58.9	not reported

Live555, Kamailio, and Lighttpd1. AFLNet remains highest on Forked-daapd. Exact Mann-Whitney testing narrows the strongest **b_abs** evidence to Pure-FTPD, Exim, Live555, and Lighttpd1. The robust coverage claim supported by the audited summaries is therefore bounded: LoopFuzz improves the closest AFLNet/ChatAFL lineage on most, but not all, primary targets.

Table 3 shows that the broader NSFuzz baseline picture is mixed. Across the eight NSFuzz targets, NSFuzz has the highest **b_abs** on LightFTP, Pure-FTPD, Exim, Kamailio, and Forked-daapd. LoopFuzz has the highest **b_abs** on bftpd, ProFTPD, and Live555. For reported **edges**, LoopFuzz is highest on seven targets, while NSFuzz is highest on Forked-daapd. Table 4 separately reports MQTT. The available LoopFuzz Mosquitto rows have higher archived **b_abs** and **edges** than the available AFLNet rows, but they are much shorter. MBFuzzer reports **branch_covered** under its own 24-hour summary, which is

not directly comparable to the AFLNet-lineage **b_abs** field.

6.2. RQ2: IPSM Escape Dynamics

On terminal **edges**, LoopFuzz attains the highest mean on all nine primary targets. The strongest same-target evidence appears on LightFTP, bftpd, ProFTPD, Pure-FTPd, Exim, and Live555, where the LoopFuzz **edges** win is significant versus both AFLNet and ChatAFL under the two-sided exact Mann-Whitney test. Kamailio, Forked-daapd, and Lighttpd1 remain positive but descriptive against at least one primary baseline. The NSFuzz auxiliary rows point in the opposite direction on Forked-daapd, where NSFuzz reports higher **edges**; on the other seven NSFuzz targets, LoopFuzz has higher reported **edges**.

6.3. RQ3: End-of-Campaign Outcome

The **b_abs** comparison is less uniform than the **edges** comparison once NSFuzz is included. In the primary three-way comparison, LoopFuzz is strongest by mean on eight of nine targets and does not lead Forked-daapd. In the auxiliary NSFuzz comparison, NSFuzz is strongest in **b_abs** on five of eight targets. This distinction matters for interpretation: the audited archive supports a strong primary-lineage state-exploration result and a broad but still selective absolute-branch-coverage result, not a blanket dominance claim over all specialized fuzzers.

6.4. Ablation: Policy Perturbations

Table 5 reports the archived policy ablations as deltas relative to the evaluated full controller. Each cell shows $\Delta \text{b_abs} / \Delta \text{edges}$. The updated archive shows a mixed rather than monotone pattern. Removing CEGR improves mean **b_abs** only on ProFTPD (+100.8) and Live555 (+1.2), while reducing it on the other seven primary targets. Removing frontier control reduces mean **b_abs** on all nine targets, but its effect on **edges** is not uniformly negative. Removing adaptive triggering improves some endpoints on Pure-FTPd, Live555, Kamailio, and Lighttpd1, while degrading others. The audited ablations therefore do not support a claim that every exposed sub-control is individually necessary across all targets.

The strongest lesson from the ablation archive is that controller switches interact with target behavior. Disabling hypothesis validation or disabling adaptive triggering can improve **b_abs** on some targets but degrade **edges** on others. This reinforces an integrated-policy interpretation rather than a clean per-module causal decomposition.

7. Discussion

The paper makes one central claim. In stateful protocol fuzzing, LLM generation is useful only when it is governed by a strict admission boundary. ChatAFL already showed that an LLM can generate protocol text [18]. LoopFuzz asks a different question: when should such output be trusted by

Table 5: Policy-ablation results relative to the evaluated full controller. The first number in each cell is final **b_abs**; the second is final **edges**. The Full column reports mean final **b_abs** / **edges**; other cells report deltas relative to that mean. Runs with archived runtime below 1400 minutes are excluded; when more than 10 valid runs remain, we keep the lowest 10 run IDs. Some ablated target-policy cells contain seven to nine eligible runs after this filter.

Target	Full	No CEGR	No frontier	No hypothesis	No adaptive trigger
LightFTP	71.6 / 213.9	-0.2 / -13.5	-0.6 / -14.5	-0.5 / -11.0	-0.6 / +1.8
bftpd	501.5 / 221.7	-3.1 / -7.5	-3.0 / -8.4	-5.2 / -5.0	-7.0 / -12.1
ProFTPD	5358.8 / 271.7	+100.8 / -4.1	-123.2 / -48.1	-17.5 / -1.3	-12.4 / +1.0
Pure-FTPd	1242.6 / 291.0	-19.2 / +1.1	-18.9 / -1.4	-0.6 / +15.9	+6.3 / +10.7
Exim	3908.5 / 128.5	-29.0 / -2.8	-40.7 / -2.8	-43.0 / -3.3	-40.6 / -8.9
Live555	3088.7 / 158.7	+1.2 / -1.5	-21.0 / -9.6	-12.2 / -4.4	+5.0 / -3.6
Kamailio	10402.7 / 127.6	-96.7 / +2.5	-121.4 / -5.0	-100.6 / +1.9	+26.9 / +5.2
Forked-daapd	2171.9 / 22.1	-104.4 / -0.9	-145.0 / +0.1	-129.9 / +0.7	-91.8 / -1.7
Lighttpd1	2156.5 / 32.5	-62.4 / -3.3	-15.9 / +1.9	-33.9 / -2.4	-27.1 / +4.1

the campaign? Our answer is closed-loop runtime admission control. It checks structure, parseability, response-grounded acceptability, state reachability, and downstream gain before an LLM suggestion can strengthen the queue. This changes the role of the LLM from queue producer to hypothesis proposer.

The admission controller and the scheduler solve different parts of the same problem. Admission control filters low-value suggestions. It does not decide where to spend the recovered budget. State-aware scheduling decides where to search. Without closed-loop intervention, however, it still depends on ordinary mutation to cross semantic bottlenecks. LoopFuzz combines the two because admission control and state-directed search reinforce each other.

The results also show why response-derived state counts must be read carefully. A campaign can accumulate superficially different rejection paths without moving toward high-value logic. LoopFuzz therefore reads state exploration together with **b_abs** and demotes states that remain unproductive over repeated selections. In this setting, productive states matter more than raw state counts. For that reason, **edges** is best interpreted as a companion diagnostic for campaign behavior, not as a substitute for absolute branch coverage.

The present method is evaluated on textual or semi-textual protocols whose message regions can be parsed cheaply and whose responses provide useful progress signals. That scope covers the benchmark suite used here and many request-response protocols of practical interest. Opaque binary or encrypted protocols require stronger parsers, auxiliary instrumentation, or richer response abstractions. Response codes alone may not expose state reachability in those settings. That extension matters, but it lies beyond the current empirical scope.

The present evaluation is also centered on campaign-control behavior rather than on vulnerability yield. We preserve crashes and queue artifacts, but we do not report new CVEs or newly confirmed bugs in this study because crash deduplication, exploitability review, and cross-target triage remain incomplete under one uniform protocol. The paper should therefore be read as evidence about campaign control and exploration quality, not as an end-to-end demonstration of stronger bug-finding yield.

Counterexample-guided local refinement is also a deliberate design choice.

Repeated unconstrained regeneration does not accumulate persistent knowledge. Local refinement turns failed executions into stored evidence. It makes runtime evidence cumulative rather than disposable. At the same time, the ablation table shows that the exposed refinement switch is not uniformly decisive on every target. The novelty therefore lies in the control interface between model output and system execution, not in prompt wording alone.

8. Threats to Validity

Internal validity: LLM-assisted fuzzing remains sensitive to generation variance, target nondeterminism, and campaign-level timing effects. We mitigate these threats by fixing the execution environment, pinning the model endpoint, using low temperatures for grammar extraction and refinement, and designing the campaign matrix around repeated 24-hour runs. Even so, plateau triggering and hypothesis refinement are history-dependent. Early differences can propagate into late-campaign variance. The study also cannot cleanly separate model variance from control-policy variance because both act within the same long-running campaigns.

External validity: The benchmark suite spans 10 implementations across seven protocol families, but the audited primary three-way comparison covers the nine off-the-shelf non-MQTT targets that contain comparable AFLNet, ChatAFL, and LoopFuzz rows. The evidence is therefore strongest for textual or semi-textual protocols whose progress is visible in responses. We do not claim unchanged transfer to opaque binary or encrypted protocols without stronger format-specific adaptors. We include auxiliary NSFuzz comparisons on eight targets and an MQTT comparison involving AFLNet, LoopFuzz, and MBFuzzer, but those rows are target- and artifact-specific rather than a unified cross-system state-machine comparison.

Construct validity: We use final `b_abs` and final `edges` as the main proxies for fuzzing effectiveness in the audited summaries. These are archive-level measurements, but neither fully captures semantic protocol progress. The MBFuzzer `branch_covered` field is reported separately because it is not the same artifact field as AFLNet-lineage `b_abs`. Our closed-loop runtime admission boundary refers to structured output checks, sampled hypothesis checks, response-grounded acceptability, state-reachability signals, and counterexample-guided refinement. It does not mean formal verification or a perfect pre-execution oracle. We also do not currently report cross-target admission summaries such as parseability pass rate, response-grounded acceptance rate, or CEGR repair success rate. The motivating semantic-drift narrative is therefore operationalized through logs rather than reduced to one global prevalence number.

Reproducibility validity: We reduce fairness ambiguity where possible. ChatAFL and LoopFuzz share the same pinned model endpoint. Each primary target is drawn from one declared result archive, and the paper states the deterministic run-selection rule used to handle short or extra rows. The provided Mosquitto rows do not support the same 24-hour three-way comparison because ChatAFL is absent and the available LoopFuzz rows are substantially shorter

than AFLNet and MBFuzzer. The ablation table is likewise an archive-level reconstruction under the same deterministic selection rule rather than a hand-picked replacement set.

9. Conclusion

We have shown that fluent protocol text is insufficient for a stateful campaign. What matters is a closed-loop runtime admission boundary that can reject semantically premature generations and refine them when they fail.

Under the audited primary comparison against AFLNet and ChatAFL, LoopFuzz demonstrates a stronger **edges** profile and a selective **b_abs** advantage. On the nine off-the-shelf targets, it achieves the highest mean final **edges** on all nine targets and the highest mean final **b_abs** on eight. The strongest same-target **b_abs** evidence appears on Pure-FTPd, Exim, Live555, and Lighttpd1; the strongest **edges** evidence appears on LightFTP, bftpd, ProFTPD, Pure-FTPd, Exim, and Live555.

The broader baseline picture is mixed. NSFuzz remains strongest on final **b_abs** for five of the eight reproduced NSFuzz targets, while LoopFuzz has the highest reported **edges** on seven of those eight. On MQTT, the available LoopFuzz rows exceed AFLNet in archived **b_abs** and **edges**, but those rows are much shorter. MBFuzzer reports **branch_covered** under 24-hour runs in its own artifact format, which is not a direct comparison with the AFLNet-lineage **b_abs** field. The archived ablations also do not support a claim that every exposed LoopFuzz sub-control is individually necessary across all targets. The evidence therefore supports a precise conclusion: closed-loop runtime admission control improves state-exploration behavior within the closest AFLNet/ChatAFL lineage and improves absolute branch coverage on most primary targets, but this study does not establish dominance over specialized external fuzzers.

Declaration of Generative AI and AI-assisted technologies in the writing process

During the preparation of this work, the authors used OpenAI Codex for language editing, LaTeX formatting, and submission-compliance checking. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

References

- [1] V.-T. Pham, M. Böhme, A. Roychoudhury, AFLNet: A greybox fuzzer for network protocols, in: 2020 IEEE 13th International Conference on Software Testing, Verification and Validation (ICST), 2020, pp. 460–465. doi:10.1109/ICST46399.2020.00062.
- [2] R. Natella, StateAFL: Greybox fuzzing for stateful network servers, Empirical Software Engineering 27 (2022). doi:10.1007/s10664-022-10233-3, art. no. 191.

- [3] J. Ba, M. Böhme, Z. Mirzamomen, A. Roychoudhury, Stateful greybox fuzzing, in: 31st USENIX Security Symposium (USENIX Security 2022), 2022, pp. 3255–3272.
- [4] F. Hu, J. Ji, H. Shu, Z. Li, T. Liu, C. Zhang, Formatted stateful greybox fuzzing of TLS server, in: 2024 IEEE Conference on Software Testing, Verification and Validation (ICST), 2024, pp. 151–160. doi:10.1109/icst60714.2024.00022.
- [5] Q. Zhang, X. Bai, X. Li, H. Duan, Q. Li, Z. Li, Resolverfuzz: Automated discovery of DNS resolver vulnerabilities with query-response fuzzing, in: 33rd USENIX Security Symposium (USENIX Security 2024), 2024, pp. 4729–4746.
- [6] X. Song, J. Wu, Y. Zeng, H. Pan, C. Zuo, Q. Zhao, S. Guo, MBFuzzer: A multi-party protocol fuzzer for MQTT brokers, in: 34th USENIX Security Symposium (USENIX Security 2025), 2025, pp. 6179–6197.
- [7] Z. Luo, J. Yu, F. Zuo, J. Liu, Y. Jiang, T. Chen, A. Roychoudhury, J. Sun, Bleem: Packet sequence oriented fuzzing for protocol implementations, in: 32nd USENIX Security Symposium (USENIX Security 2023), 2023, pp. 4481–4498.
- [8] F. Wu, Z. Luo, Y. Zhao, Q. Du, J. Yu, R. Peng, H. Shi, Y. Jiang, Logos: Log guided fuzzing for protocol implementations, in: Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, 2024, pp. 1720–1732. doi:10.1145/3650212.3680394.
- [9] N. Redini, A. Continella, D. Das, G. D. Pasquale, N. Spahn, A. Machiry, A. Bianchi, C. Kruegel, G. Vigna, DIANE: Identifying fuzzing triggers in apps to generate under-constrained inputs for iot devices, in: 2021 IEEE Symposium on Security and Privacy (SP), 2021, pp. 484–500. doi:10.1109/SP40001.2021.00066.
- [10] Y. Dong, S. M. M. Rashid, A. Ranjbar, T. Wu, S. R. Hussain, M. S. Mahmud, A. A. Ishtiaq, K. Tu, T. Yang, CoreCrisis: Threat-guided and context-aware iterative learning and fuzzing of 5g core networks, in: 34th USENIX Security Symposium (USENIX Security 2025), 2025, pp. 5287–5306.
- [11] M. Böhme, V.-T. Pham, M.-D. Nguyen, A. Roychoudhury, Directed greybox fuzzing, in: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, 2017, pp. 2329–2344. doi:10.1145/3133956.3134020.
- [12] A. Herrera, H. Gunadi, S. Magrath, M. Norrish, M. Payer, A. L. Hosking, Seed selection for successful fuzzing, in: Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis, 2021, pp. 230–243. doi:10.1145/3460319.3464829.
- [13] M. Wu, L. Jiang, J. Xiang, Y. Huang, H. Cui, L. Zhang, Y. Zhang, One fuzzing strategy to rule them all, in: Proceedings of the 44th International Conference on Software Engineering, 2022, pp. 1634–1645. doi:10.1145/3510003.3510174.
- [14] X. Zhu, M. Böhme, Regression greybox fuzzing, in: Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security

- rity, 2021, pp. 2169–2182. doi:10.1145/3460120.3484596.
- [15] J. Ba, G. J. Duck, A. Roychoudhury, Efficient greybox fuzzing to detect memory errors, in: Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering, 2022, pp. 1–12. doi:10.1145/3551349.3561161.
 - [16] R. Natella, V.-T. Pham, ProFuzzBench: A benchmark for stateful protocol fuzzing, in: Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis, 2021, pp. 662–665. doi:10.1145/3460319.3469077.
 - [17] A. Andronidis, C. Cadar, SnapFuzz: high-throughput fuzzing of network applications, in: Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis, 2022, pp. 340–351. doi:10.1145/3533767.3534376.
 - [18] R. Meng, M. Mirchev, M. Böhme, A. Roychoudhury, Large language model guided protocol fuzzing, in: 31st Annual Network and Distributed System Security Symposium (NDSS 2024), 2024, pp. 1–17. doi:10.14722/ndss.2024.24556.
 - [19] C. S. Xia, M. Paltenghi, J. L. Tian, M. Pradel, L. Zhang, Fuzz4All: Universal fuzzing with large language models, in: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, 2024, pp. 1–13. doi:10.1145/3597503.3639121.
 - [20] J. Wang, L. Yu, X. Luo, LLMIF: Augmented large language model for fuzzing IoT devices, in: 2024 IEEE Symposium on Security and Privacy (SP), 2024, pp. 881–896. doi:10.1109/sp54263.2024.00211.
 - [21] X. Ma, L. Luo, Q. Zeng, From one thousand pages of specification to unveiling hidden bugs: Large language model assisted fuzzing of matter IoT devices, in: 33rd USENIX Security Symposium (USENIX Security 2024), 2024, pp. 4783–4800.
 - [22] D. Wang, Y. Li, Z. Zhang, K. Chen, CarpetFuzz: Automatic program option constraint extraction from documentation for fuzzing, in: 32nd USENIX Security Symposium (USENIX Security 2023), 2023, pp. 1919–1936.
 - [23] Y. Lyu, Y. Xie, P. Chen, H. Chen, Prompt fuzzing for fuzz driver generation, in: Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security, 2024, pp. 3793–3807. doi:10.1145/3658644.3670396.
 - [24] Y. Deng, C. S. Xia, H. Peng, C. Yang, L. Zhang, Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models, in: Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, 2023, pp. 423–435. doi:10.1145/3597926.3598067.
 - [25] Y. Deng, C. S. Xia, C. Yang, S. Zhang, S. Yang, L. Zhang, Large language models are edge-case generators: Crafting unusual programs for fuzzing deep learning libraries, in: Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, 2024, pp. 70:1–70:13. doi:10.1145/3597503.3623343.

- [26] P. Srivastava, M. Payer, Gramatron: effective grammar-aware fuzzing, in: *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2021, pp. 244–256. doi:10.1145/3460319.3464814.
- [27] J. Fu, J. Liang, Z. Wu, M. Wang, Y. Jiang, Griffin: Grammar-free dbms fuzzing, in: *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, 2022, pp. 1–12. doi:10.1145/3551349.3560431.
- [28] X. Feng, R. Sun, X. Zhu, M. Xue, S. Wen, D. Liu, S. Nepal, Y. Xiang, Snipuzz: Black-box fuzzing of IoT firmware via message snippet inference, in: *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, 2021, pp. 337–350. doi:10.1145/3460120.3484543.
- [29] C. Salls, C. Jindal, J. Corina, C. Kruegel, G. Vigna, Token-level fuzzing, in: *30th USENIX Security Symposium (USENIX Security 2021)*, 2021, pp. 2795–2809.
- [30] A. Bulekov, B. Das, S. Hajnoczi, M. Egele, No grammar, no problem: Towards fuzzing the Linux kernel without system-call descriptions, in: *30th Annual Network and Distributed System Security Symposium (NDSS 2023)*, 2023, pp. 1–16. doi:10.14722/ndss.2023.24688.
- [31] A. Fioraldi, D. C. D’Elia, D. Balzarotti, The use of likely invariants as feedback for fuzzers, in: *30th USENIX Security Symposium (USENIX Security 2021)*, 2021, pp. 2829–2846.
- [32] C. Myung, G. Lee, B. Lee, MundoFuzz: Hypervisor fuzzing with statistical coverage testing and grammar inference, in: *31st USENIX Security Symposium (USENIX Security 2022)*, 2022, pp. 1257–1274.
- [33] S. Qin, F. Hu, Z. Ma, B. Zhao, T. Yin, C. Zhang, NSFuzz: Towards efficient and state-aware network service fuzzing, *ACM Transactions on Software Engineering and Methodology* 32 (2023) 160:1–160:26. doi:10.1145/3580598.
- [34] Y. Zhang, D. Fang, P. Liu, L. Xi, X. Lu, X. Chen, S. Si, L. Sun, MSGFuzzer: Message sequence guided industrial robot protocol fuzzing, in: *2024 IEEE Conference on Software Testing, Verification and Validation (ICST)*, 2024, pp. 140–150. doi:10.1109/icst60714.2024.00021.
- [35] K. Kim, T. Kim, E. Warraich, B. Lee, K. R. B. Butler, A. Bianchi, D. J. Tian, FuzzUSB: Hybrid stateful fuzzing of USB gadget stacks, in: *2022 IEEE Symposium on Security and Privacy (SP)*, 2022, pp. 2212–2229. doi:10.1109/sp46214.2022.9833593.
- [36] P. Fiterau-Brostean, B. Jonsson, K. Sagonas, F. Taquist, DTLs-Fuzzer: A DTLs protocol state fuzzer, in: *2022 IEEE Conference on Software Testing, Verification and Validation (ICST)*, 2022, pp. 456–458. doi:10.1109/icst53961.2022.00051.
- [37] K. Sagonas, T. Typaldos, EDHOC-Fuzzer: An EDHOC protocol state fuzzer, in: *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2023, pp. 1495–1498. doi:10.1145/3597926.3604922.

- [38] Q. Zhang, Y. Chu, Y. Shen, J. Liu, H. Shi, Y. Jiang, W. Chang, Tron: Fuzzing Linux network stack via protocol-system call payload synthesis, in: 2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE), 2025, pp. 3227–3238. doi:10.1109/ase63991.2025.00266.
- [39] H. Kim, M. O. Ozmen, A. Bianchi, Z. B. Celik, D. Xu, PGFUZZ: Policy-guided fuzzing for robotic vehicles, in: 28th Annual Network and Distributed System Security Symposium (NDSS 2021), 2021, pp. 1–18. doi:10.14722/ndss.2021.24096.
- [40] Y. Aafer, W. You, Y. Sun, Y. Shi, X. Zhang, H. Yin, Android smarttvs vulnerability discovery via log-guided fuzzing, in: 30th USENIX Security Symposium (USENIX Security 2021), 2021, pp. 2759–2776.
- [41] T. Scharnowski, N. Bars, M. Schloegel, E. Gustafson, M. Muench, G. Vigna, C. Kruegel, T. Holz, A. Abbasi, Fuzzware: Using precise MMIO modeling for effective firmware fuzzing, in: 31st USENIX Security Symposium (USENIX Security 2022), 2022, pp. 1239–1256.
- [42] B. Zhao, Z. Li, S. Qin, Z. Ma, M. Yuan, W. Zhu, Z. Tian, C. Zhang, StateFuzz: System call-based state-aware linux driver fuzzing, in: 31st USENIX Security Symposium (USENIX Security 2022), 2022, pp. 3273–3289.
- [43] D. Ryu, G. Kim, D. Lee, S. Kim, S. Bae, J. Rhee, T. Kim, Differential fuzzing for data distribution service programs with dynamic configuration, in: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, 2024, pp. 807–818. doi:10.1145/3691620.3695073.
- [44] T. Trippel, K. G. Shin, A. Chernyakhovsky, G. Kelly, D. Rizzo, M. Hicks, Fuzzing hardware like software, in: 31st USENIX Security Symposium (USENIX Security 2022), 2022, pp. 3237–3254.
- [45] M. Beckmann, J. Steffan, Coverage-guided fuzzing of embedded systems leveraging hardware tracing, in: Computer Security – ESORICS 2023, 2023, pp. 362–378. doi:10.1007/978-3-031-25460-4_21.
- [46] Z. Du, Y. Li, Y. Liu, B. Mao, WindRanger: A directed greybox fuzzer driven by deviation basic blocks, in: Proceedings of the 44th International Conference on Software Engineering, 2022, pp. 2440–2451. doi:10.1145/3510003.3510197.
- [47] P. Lin, P. Wang, X. Zhou, W. Xie, X. Ren, K. Lu, When control flows deviate: Directed grey-box fuzzing with probabilistic reachability analysis, in: Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering, 2025, pp. 713–725. doi:10.1109/ASE63991.2025.00065.
- [48] Y. Chen, R. Zhong, H. Hu, H. Zhang, Y. Yang, D. Wu, W. Lee, One Engine to Fuzz ’em All: Generic language processor testing with semantic validation, in: 2021 IEEE Symposium on Security and Privacy (SP), 2021, pp. 642–658. doi:10.1109/SP40001.2021.00071.
- [49] C. Zhang, X. Lin, Y. Li, Y. Xue, J. Xie, H. Chen, X. Ying, J. Wang, Y. Liu, APICraft: Fuzz driver generation for closed-source sdk libraries,

- in: 30th USENIX Security Symposium (USENIX Security 2021), 2021, pp. 2811–2828.
- [50] J. Jiang, H. Xu, Y. Zhou, RULF: Rust library fuzzing via api dependency graph traversal, in: Proceedings of the 36th IEEE/ACM International Conference on Automated Software Engineering, 2021, pp. 581–592. doi:10.1109/ASE51524.2021.9678813.
 - [51] J. Choi, D. Kim, S. Kim, G. Grieco, A. Groce, S. K. Cha, SMARTIAN: Enhancing smart contract fuzzing with static and dynamic data-flow analyses, in: Proceedings of the 36th IEEE/ACM International Conference on Automated Software Engineering, 2021, pp. 227–239. doi:10.1109/ASE51524.2021.9678888.
 - [52] Z. Zhou, Y. Yang, S. Wu, Y. Huang, B. Chen, X. Peng, Magneto: A step-wise approach to exploit vulnerabilities in dependent libraries via llm-empowered directed fuzzing, in: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, 2024, pp. 1633–1644. doi:10.1145/3691620.3695531.
 - [53] D. Xie, Y. Li, M. Kim, H. V. Pham, L. Tan, X. Zhang, M. W. Godfrey, DocTer: documentation-guided fuzzing for testing deep learning API functions, in: Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis, 2022, pp. 176–188. doi:10.1145/3533767.3534220.
 - [54] M. Schloegel, N. Bars, N. Schiller, L. Bernhard, T. Scharnowski, A. Crump, A. Ale-Ebrahim, N. Bissantz, M. Muench, T. Holz, SoK: Prudent evaluation practices for fuzzing, in: 2024 IEEE Symposium on Security and Privacy (SP), 2024, pp. 1974–1993. doi:10.1109/sp54263.2024.00137.
 - [55] M. Böhme, L. Szekeres, J. Metzman, On the reliability of coverage-based fuzzer benchmarking, in: Proceedings of the 44th International Conference on Software Engineering, 2022, pp. 1621–1633. doi:10.1145/3510003.3510230.
 - [56] C. Poncelet, K. K. Sagonas, N. Tsiftes, So many fuzzers, so little time: Experience from evaluating fuzzers on the contiki-ng network stack, in: Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering, 2022, pp. 1–13. doi:10.1145/3551349.3556946.